

PostgreSQL shines in scenarios that require data integrity, custom data types, advanced indexing, geospatial support (PostGIS), and horizontal scaling options. It's a top choice for financial systems, scientific research, analytics platforms, and enterprise-grade applications that need to handle large volumes of data with accuracy and control. With a strong academic foundation and decades of active development, PostgreSQL offers not just features, but future-proofing — thanks to its modular architecture and thriving plugin ecosystem.

Head-to-head comparison: Key technical aspects

Architecture and design philosophy:

MySQL and MariaDB use a pluggable storage engine model, so different engines can be used for different needs. Their default engine, InnoDB, supports transactions and full ACID compliance, but developers can switch to engines like MyISAM (optimised for read-heavy legacy workloads) or Aria (a MariaDB-specific engine designed for crash recovery). In contrast, PostgreSQL uses a unified monolithic architecture that is less modular but highly extensible, allowing the creation of custom data types, operators, indexing methods, and procedural functions. This approach favours predictable performance and strict SQL standard compliance, making PostgreSQL a preferred choice for enterprise-grade systems where consistency is key.

Performance and concurrency:

MySQL and MariaDB are generally faster for simple read/write operations, especially in web applications with low-complexity transactions, due to lightweight configurations and aggressive caching. However, when dealing with complex queries involving multiple joins, aggregations, and recursion, PostgreSQL outperforms thanks to its sophisticated query planner, support for Common Table

Expressions (CTEs), window functions, and advanced indexing methods like GIN and BRIN. For concurrency, PostgreSQL leverages Multiversion Concurrency Control (MVCC) more effectively, enabling multiple readers and writers to interact without blocking each other. MySQL/MariaDB also use MVCC via InnoDB but tend to fall back to traditional locking in edge cases, which can impact performance in high-concurrency environments.

Data integrity and ACID

compliance: All three databases provide full ACID compliance when transactional engines like InnoDB (for MySQL/MariaDB) or PostgreSQL's native system are used. However, PostgreSQL stands out with its rigorous implementation of isolation levels (including Serializable and Repeatable Read) and support for nested transactions through SAVEPOINT, allowing for more controlled rollbacks and finer transaction management. MySQL and MariaDB, while compliant, require careful engine selection, as switching to non-transactional engines like MyISAM compromises ACID guarantees.

Data types and JSON support:

PostgreSQL excels in data type richness. Beyond standard SQL types like INTEGER, FLOAT, VARCHAR, it supports native arrays, UUIDs, geometric types, IP addresses, and even user-defined types, making it ideal for complex data models. Its JSON support is industry-leading, offering both JSON and JSONB (binary JSON), which allows indexing, deep querying, and seamless integration with SQL queries. In contrast, MySQL and MariaDB offer JSON data types and functions, but their JSON features are less powerful, often lacking advanced querying and indexing support, making PostgreSQL the top choice for semi-structured data.

Replication and scalability:

Replication is vital for high availability and distributed systems. MySQL supports asynchronous and semi-synchronous replication, making it suitable for simple master-slave setups. MariaDB enhances this with multi-source replication (replicating from multiple masters) and Galera Cluster for true synchronous replication and high availability. PostgreSQL supports both physical (streaming) and logical replication, as well as bidirectional replication (BDR) via community-supported extensions, allowing more flexible replication strategies. For scalability, all three support vertical scaling (adding CPU/RAM), but for horizontal scaling, PostgreSQL uses extensions like Citus for distributed workloads, whereas MariaDB offers the Spider storage engine to support sharding, giving developers multiple options based on architecture needs.

Extensibility and features:

PostgreSQL is widely recognised for its deep extensibility. It allows custom extensions like PostGIS (for geospatial data), pg_partman (for partition management), and pluggable procedural languages (like PL/pgSQL, Python, Perl, etc). It supports event triggers, custom background workers, and a wide range of performance tuning tools. MariaDB brings its own innovation, such as dynamic columns (to store varied schema data within a single table), virtual/generated columns, and ColumnStore for analytical (OLAP) queries. MySQL, while more conservative in features, remains reliable and widely supported, especially in traditional LAMP stack environments. All three support stored procedures, triggers, views, and event schedulers, but PostgreSQL provides greater depth and control.

Licensing: Licensing can impact long-term project decisions. PostgreSQL uses a permissive licence